

BOOSTING SEMANTIC SIMILARITY WITH TRANSFORMERS: A MULTI-MODEL STUDY

Enrol. No. (s) – 9923103039, 9923103046, 9923103056

Name (s) – Pragya Varshney, Aryan Jain, Astha Bansal

Name of supervisor (s) – Dr. Akanksha Mehndiratta



May – 2026

Submitted in partial fulfillment of the Degree of

Bachelor of Technology

in

Computer Science Engineering

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND
INFORMATION TECHNOLOGY**

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

CERTIFICATE

This is to certify that the work titled “Boosting Semantic Similarity with Transformers: A Multi-Model Comparative Study” submitted by Pragya Varshney, Aryan Jain and Astha Bansal of B.Tech of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of any other degree or diploma.

Signature of Supervisor

Dr. Akanksha Mehndiratta

Signature of HOD

Dr. Shikha Mehta

Date: May 06, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and beliefs, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma from a university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place: Jaypee Institute of Information Technology

Date: May 06, 2026

Name: Pragya Varshney

Enrolment No.: 9923103039

Name: Aryan Jain

Enrolment No.: 9923103046

Name: Astha Bansal

Enrolment No.: 9923103056

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to our project advisor and supervisor, Dr. Akanksha Mehndiratta, Dept. of Computer Science & Engineering and Information Technology, Jaypee Institute of Information Technology, India for her generous guidance, help and useful suggestions.

I also wish to extend my thanks to supporting faculty and other classmates for their insightful comments and constructive suggestions to improve the quality of this project work.

Name of Students	Enrollment	Signature
Pragya Varshney	9923103039	
Aryan Jain	9923103046	
Astha Bansal	9923103056	

ABSTRACT

Semantic similarity is a fundamental task in Natural Language Processing (NLP) that focuses on measuring how closely two pieces of text convey the same meaning. With the advancement of deep learning, transformer-based models have significantly improved the ability of machines to understand contextual relationships between words and sentences. This project explores how transformer architectures can be utilized to compute semantic similarity between textual responses, with Automated Short Answer Grading (ASAG) used as the primary application domain.

In this work, transformer-based language models are employed to generate contextual embeddings for both reference answers and student responses. These embeddings are then compared using similarity measures to estimate how semantically close a student's answer is to the expected answer. The effectiveness of this approach is evaluated using the Mohler dataset, where student answers are compared against instructor-provided reference answers. Performance is assessed using evaluation metrics such as Pearson Correlation, Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and Quadratic Weighted Kappa (QWK) to measure agreement between automated scores and human-assigned grades.

The initial experiments focus on embedding-based similarity using transformer models such as Sentence-BERT, providing insights into how contextual representations capture semantic meaning in short textual responses. The study aims to analyze the capability of transformer models in understanding semantic relationships and their effectiveness in practical educational applications. Future stages of the project will extend the analysis by experimenting with additional transformer architectures such as Pythia and T5, along with improved preprocessing and cross-dataset evaluation.

TABLE OF CONTENTS

LIST OF TABLES.....	i
LIST OF FIGURES	ii
LIST OF ABBREVIATIONS	iii
CHAPTER 1: INTRODUCTION.....	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Social & Environmental Relevance	2
1.4 Timeline.....	3
1.5 Scope	3
CHAPTER 2: BACKGROUND STUDY	5
2.1 Overview of Existing Work.....	5
2.2 Summary Table.....	10
2.3 Limitations of Existing Methods	11
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY	12
3.1 Proposed Methodology.....	12
3.2. Tool and Technique Selection.....	17
CHAPTER 4: IMPLEMENTATION AND TESTING	19
4.1 Experiment Setup	19
4.2 Flow Charts and Test Metrics.....	22
CHAPTER 5: RESULTS AND DISCUSSION	25
5.1 Result analysis.....	25
5.2 Performance Evaluation	30
Chapter 6: CONCLUSION AND FUTURE SCOPE	35
6.1 Summary of Work and Achievements	35
6.2 Impact on Society, Environmental Sustainability, and Ethical Considerations	35
6.3 Limitations of the Work.....	36
6.4 Future Scope.....	37
REFERENCES.....	38
APPENDICES	39
Code Snippets.....	39
ANNEXURE	40
Sustainability Statement	40
Team Roles and Contributions	40

LIST OF TABLES

Table 1- Embeddings for Automatic Short Answer Grading: A Scoping Review	5
Table 2- Advantages & Disadvantages of different types of embeddings	6
Table 3-Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.....	7
Table 4- Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling	8
Table 5-Scalable Sentence Encoders from Pre-trained Text-to-Text Models	9
Table 6- Comparison of transformer models performance across the three experiments.....	31
Table 7-Comparison of datasets performance across the three experiments	32

LIST OF FIGURES

Figure 1-BERT vs SBERT Pipeline	7
Figure 2:Overall pipeline of our system	15
Figure 3:System architecture diagram for running transformer-based STS task.....	22
Figure 4: Work Flow for model without task specific training.....	23
Figure 5: Work Flow of model with task specific training	23
Figure 6: Pearson Correlation Comparison for Experiment 1	25
Figure 7: RMSE Comparison for Experiment 1	25
Figure 8: QWK Comparison for Experiment 1.....	26
Figure 9: Pearson Heatmap for Experiment 1.....	26
Figure 10: Pearson Comparison for Experiment 2	27
Figure 11: RMSE Comparison for Experiment 2	27
Figure 12: QWK Comparison for Experiment 2.....	27
Figure 13: Pearson Heatmap for Experiment 2.....	28
Figure 14: Pearson Comparison for Experiment 3	28
Figure 15: RMSE Comparison for Experiment 3	29
Figure 16: QWK Comparison for Experiment 3.....	29
Figure 17: Pearson Heatmap for Experiment 3.....	29

LIST OF ABBREVIATIONS

- **ASAG** -Automated Short Answer Grading.
- **NLP** – Natural Language Processing
- **STS**- Semantic Textual Similarity
- **LLM**- Large Language Model
- **ACL**- Association for Computational Linguistics
- **PMLR** – Proceedings of Machine Learning Research
- **IEEE**- Institute of Electrical and Electronics Engineers
- **BERT** – Bidirectional Encoder Representations from Transformers
- **SBERT** – Sentence-BERT
- **T5** – Text-to-Text Transfer Transformer
- **RMSE** – Root Mean Square Error
- **MAE** – Mean Absolute Error
- **QWK** – Quadratic Weighted Kappa

CHAPTER 1: INTRODUCTION

1.1 Background

In recent years, the rapid growth of digital communication, online education platforms, and large-scale textual data has created a strong demand for intelligent systems capable of understanding and analyzing human language automatically.

One of the most important challenges in NLP is determining the degree of similarity in meaning between two pieces of text. This concept is known as semantic similarity. Semantic similarity refers to the process of measuring how closely two words, phrases, or sentences are related in terms of their meaning rather than their exact wording. These include information retrieval systems, question answering systems, text summarization tools, plagiarism detection systems, and conversational agents.

Among these applications, one of the most significant and practical uses of semantic similarity is in automated answer grading systems used in educational environments. In automated answer grading, the system typically compares a student's response with a reference or model answer provided by the instructor. Instead of relying solely on keyword matching, grading systems should analyze the semantic similarity between the student's answer and the expected answer. If the student expresses the correct concept using different wording, the system should still recognize the similarity in meaning and assign an appropriate score. This capability significantly improves the fairness and effectiveness of automated assessment systems.

The motivation behind this project is to explore and compare different NLP-based transformer approaches for measuring semantic similarity and to evaluate their suitability for automated answer grading systems. This study aims to identify the strengths and limitations of different approaches and contribute toward the development of more accurate, efficient, and reliable automated evaluation systems for educational applications.

1.2 Problem Statement

With the rapid growth of online education platforms and digital learning environments, educational institutions are increasingly required to evaluate a large number of student responses efficiently. Traditional evaluation methods rely on manual grading performed by instructors, especially for descriptive or short-answer questions. While manual grading allows instructors to assess conceptual understanding, it is often time-consuming, labor-intensive, and difficult to scale when dealing with large classes or online assessments. In addition, manual grading may introduce subjectivity and inconsistencies, as different evaluators may interpret student responses differently.

Automated Short Answer Grading (ASAG) systems have emerged as a potential solution to these challenges by enabling the automatic evaluation of student responses using computational techniques. These systems aim to analyze the content of a student's answer and determine how closely it matches the expected or reference answer. However, one of the key challenges in automated grading is to accurately understand the semantic meaning of natural language responses. Students may express correct concepts using different words, sentence structures, or paraphrased explanations. Simple keyword matching or rule-based approaches often fail to capture the underlying meaning of such responses, leading to inaccurate grading. Natural Language Processing (NLP) techniques have been widely explored to address this problem by enabling machines to analyze and interpret textual data. Recent advances in deep learning have significantly improved semantic understanding through transformer-based language models. These models use attention mechanisms to capture contextual relationships between words and sentences, allowing for more accurate representation of textual meaning. Transformer architectures can be broadly categorized into encoder models, decoder models, and encoder–decoder models, each with different capabilities and applications in NLP tasks. Although these models have demonstrated strong performance in various NLP tasks, their effectiveness in measuring semantic similarity for automated answer grading may differ due to variations in architecture and representation of learning capabilities. Therefore, the primary problem addressed in this study is the need to evaluate and compare different transformer architectures—encoder, decoder, and encoder–decoder models—for measuring semantic similarity in student responses. By conducting a comparative study of representative models from each category, this research aims to identify the most effective approach for accurately capturing semantic meaning and improving the reliability of automated answer grading systems in modern educational environments.

Objectives of the Study

The primary objectives of this study are:

1. To analyze how transformer-based models capture and encode semantic meaning in sentences.
2. To compare the performance of different transformer architectures in semantic similarity tasks.
3. To analyze model performance across multiple datasets.
4. To evaluate the impact of fine-tuning on semantic similarity performance.

1.3 Social & Environmental Relevance

The development of transformer-based semantic similarity systems has significant social relevance, particularly in the field of education. With the rapid expansion of online learning platforms and digital classrooms, educational institutions are required to evaluate a large number of student responses

efficiently. Automated short-answer grading systems can reduce the workload of educators by providing fast and consistent evaluation of student answers. This enables instructors to focus more on teaching and personalized student support rather than manual assessment tasks.

In addition, such systems promote fairness and consistency in grading. Unlike manual evaluation, which may vary between different evaluators, automated systems provide standardized assessment based on semantic understanding. This helps in reducing bias and ensures that students are evaluated based on the correctness of their concepts rather than the exact wording of their answers.

From a broader perspective, semantic similarity techniques are also applicable in various domains such as information retrieval, customer support systems, and intelligent search engines. These applications improve accessibility to information and enhance user experience by providing more relevant and meaningful results.

In terms of environmental relevance, automated systems reduce the dependency on paper-based evaluation methods, contributing to a reduction in paper usage and physical resource consumption. Digital evaluation systems also support remote learning environments, minimizing the need for physical infrastructure and travel, which indirectly contributes to energy conservation and reduced carbon footprint.

Thus, the proposed system not only improves efficiency in educational assessment but also supports sustainable and scalable digital solutions.

1.4 Timeline

- **Week 1** – Evaluation of problem statement and background study.
- **Week 2-3** - Literature review and evaluation of limitations of existing studies.
- **Week 4-5** – Dataset collection, evaluation and preprocessing.
- **Week 6** – Experiment 1- Similarity based baseline implementation and analysis of results.
- **Week 7-8** – Experiment 2- Regression based implementation and comparative analysis of results.
- **Week 9-10** – Experiment 3- Fine tuning of embedder models and analysis of subsequent results.
- **Week 11** – Overall analysis of results with answers to comparative research.
- **Week 12** – Future scope and report writing.

1.5 Scope

The scope of this study is focused on analyzing and improving semantic similarity techniques using transformer-based models for automated short-answer grading. The system is designed to evaluate the

similarity between student answers and reference answers using different transformer architectures, including encoder-based, decoder-based, and encoder–decoder models.

The scope of the project is limited to textual data and short-answer responses. The system evaluates semantic similarity at the sentence level and does not consider other modalities such as images, audio, or long-form essay evaluation. Additionally, the system relies on the availability of reference answers and does not address fully open-ended or subjective grading scenarios.

While the study explores multiple transformer models and evaluates their performance, it does not focus on developing new transformer architectures. Instead, it emphasizes the application, comparison, and analysis of existing models for the specific task of automated grading.

The findings of this study can be extended to other natural language processing applications such as plagiarism detection, question answering systems, and document similarity analysis, making it relevant for future research and development in semantic understanding systems.

CHAPTER 2: BACKGROUND STUDY

2.1 Overview of Existing Work

Automated Short Answer Grading (ASAG) is an important application of natural language processing that aims to evaluate students' descriptive answers automatically. Early ASAG systems used rule-based or keyword-matching techniques, comparing student responses with predefined concepts. While simple and efficient, these methods struggled to recognize correct answers expressed in different words. To address this limitation, modern approaches use embeddings—numerical vector representations of text—to capture semantic meaning.

Recent research highlights the effectiveness of transformer-based models like Sentence-BERT, Pythia, and T5 in improving semantic similarity tasks central to ASAG. Sentence-BERT enables efficient comparison of sentence meanings using a siamese architecture, while Pythia offers insights into how large language models develop linguistic understanding across training stages. T5, although flexible and powerful, requires careful tuning and higher computational resources. Despite these advancements, challenges remain, including high computational costs and variability in model performance across domains.

The research articles studied during the semester for the development of this study are as follows:

Table 1- Embeddings for Automatic Short Answer Grading: A Scoping Review

Title	Embeddings for Automatic Short Answer Grading: A Scoping Review
Author	Marko Putnikovic & Jelena Jovanovic
Journal Article	IEEE Transactions on Learning Technologies, 2023
Summary	Automated Short Answer Grading (ASAG) is a task in Natural Language Processing (NLP) that focuses on automatically evaluating students' short answer responses. While manual grading allows instructors to evaluate conceptual understanding, it becomes very much time consuming and difficult to scale when dealing with large numbers of students on online learning platforms. ASAG systems aim to address this challenge by automatically analyzing student responses and assigning grades based on their semantic similarity to instructor provided reference answers. Early ASAG methods relied on rule-based and non-linear approaches such as concept mapping and information retrieval techniques. Although these approaches are relatively simple to implement, they have several limitations. They often fail to

recognize correct answers when students use different words or sentence structures to express the same and correct idea.

Embeddings and types

Embeddings are numerical vector representations of textual data that capture semantic and syntactic relationships between words and sentences. So instead of representing words as discrete symbols, embedding techniques map them into continuous vector spaces where semantically similar words or sentences appear closer to each other.

Word embeddings represent words as numerical vectors learned from large text datasets based on how frequently words appear together. Models like Word2Vec and GloVe use this approach, but they assign the same vector to a word regardless of the context in which it appears. To overcome this limitation, contextual embeddings generate different representations for a word depending on the surrounding text, allowing models such as BERT to better understand the meaning of words in different situations. Sentence embeddings extend this idea by representing entire sentences as single vectors that capture the overall meaning of the text, making them useful for tasks like semantic similarity, information retrieval, and automated answer grading.

Table 2- Advantages & Disadvantages of different types of embeddings

Embeddings	Advantages	Disadvantages
Word embeddings	Simple and computationally efficient. Perform well for many NLP tasks.	Cannot capture multiple meaning. Cannot represent phrases or multi-word expressions. Struggle with out-of-vocabulary words.
Contextual embeddings	Generate different vectors depending on context Widely used in modern NLP Large pretrained vocabularies	More computationally expensive Require high resources for training and fine-tuning Do not use external lexical knowledge resources.
Sentence embeddings	Good for representing full phrases and sentences	Computationally more complex Unclear if they represent meaning better than simple combinations of word embeddings .

Table 3-Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

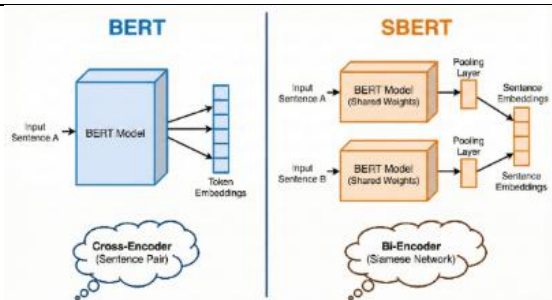
Title	Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks
Author	Nils Reimers & Iryna Gurevych
Conference Proceeding Article	<i>Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing</i>
Weblink	https://aclanthology.org/D19-1410.pdf
Summary	 Figure 1-BERT vs SBERT Pipeline Sentence-BERT (SBERT) is a transformer-based model designed to generate meaningful sentence embeddings for semantic similarity tasks. It was proposed by Reimers and Gurevych as an extension of the BERT architecture to improve the efficiency of sentence similarity computation. In the original BERT model, two sentences must be processed together to determine their similarity, which makes large-scale comparison computationally expensive. SBERT addresses this limitation by introducing a siamese network architecture, where two identical BERT models share parameters and encode sentences independently. In this architecture, each sentence is passed through the BERT encoder to produce contextual token embeddings. A pooling operation, such as mean pooling or the [CLS] token representation, is then applied to obtain a fixed-length vector representing the entire sentence. Because sentences are encoded separately, their embeddings can be stored and compared efficiently using similarity measures such as cosine similarity. SBERT has been widely used in tasks such as semantic textual similarity, information retrieval, sentence clustering, and question answering. Its ability to produce high-quality sentence embeddings makes it particularly suitable for applications where understanding the semantic meaning of sentences is important. In this project, SBERT is used to generate embeddings for both the reference answer and the student answer, and the similarity between these embeddings is calculated using cosine similarity to evaluate how closely the student response matches the expected answer.

Table 4- Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling

Title	Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling
Author	Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, Oskar van der Wal
Conference Proceeding Article	<i>Biderman, Stella, et al. "Pythia: A suite for analyzing large language models across training and scaling." International conference on machine learning. PMLR, 2023.</i>
Summary	Biderman et al. (2023) introduced Pythia, a comprehensive suite of transformer-based language models developed to systematically analyze the behavior, learning dynamics, and scaling properties of large language models. Unlike traditional transformer models that are typically released only in their final trained form, the Pythia suite provides multiple intermediate checkpoints throughout the training process. This enables fine-grained analysis of how linguistic, syntactic, and semantic knowledge emerges as the model scales in terms of both parameters and training data. The Pythia models are built on the autoregressive transformer architecture and are trained on large-scale textual corpora, making them capable of capturing deep contextual relationships between tokens and sequences. A key contribution of this work is its focus on interpretability and transparency, allowing researchers to study how representations evolve and how models progressively acquire the ability to understand semantic similarity, contextual dependencies, and abstract language patterns. In the context of semantic similarity tasks, such as those involved in Automated Short Answer Grading (ASAG), transformer models like Pythia are particularly relevant due to their ability to generate rich contextual embeddings and perform implicit reasoning over textual inputs. Their capacity to model nuanced semantic relationships between a reference answer and a student response makes them suitable for evaluating answer correctness beyond simple lexical overlap. Furthermore, the availability of multiple model scales and training checkpoints in Pythia allows for comparative analysis of performance across different levels of model complexity. This is especially useful in this project, where the objective is to study how transformer-based models perform under different experimental settings, including embedding-based similarity and classification-based approaches.

Table 5-Scalable Sentence Encoders from Pre-trained Text-to-Text Models

Title	Scalable Sentence Encoders from Pre-trained Text-to-Text Models
Author	Jianmo Ni, Gustavo Hernández Ábrego, Noah Constant, Ji Ma, Keith B. Hall, Daniel Cer, Yinfei Yang
Conference Proceeding Article	Ni, Jianmo, et al. "Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models." <i>Findings of the association for computational linguistics: ACL 2022</i> .
Weblink	https://aclanthology.org/2022.findings-acl.146.pdf
Summary	The paper on T5 investigates how a unified text-to-text framework can be adapted to generate sentence embeddings for semantic similarity tasks. Unlike traditional encoder-only models that directly produce embeddings, T5 reformulates the problem by converting input sentences into a structured format and training the model to produce representations that preserve semantic relationships. The study explores multiple strategies, including using encoder outputs, decoder outputs, and fine-tuning objectives specifically designed for similarity learning. A key contribution of the work is the adaptation of T5 for embedding generation, which is not its original purpose. The authors experiment with different pooling methods and training techniques to improve the quality of sentence representations. They also evaluate the model on standard semantic textual similarity (STS) benchmarks and compare its performance with strong encoder-based baselines. Results show that while T5 can achieve competitive performance, it often requires careful fine-tuning and task-specific design choices to match or exceed models specifically built for embedding tasks. The paper further highlights that T5's encoder-decoder architecture provides flexibility, allowing it to model complex relationships between sentence pairs. However, this flexibility comes at the cost of higher computational complexity and less efficiency compared to simpler encoder-only models. Additionally, the study emphasizes that performance varies across datasets, reinforcing the idea that different transformer architectures behave differently depending on domain characteristics and data distribution. Overall, the research demonstrates that T5 is a powerful and versatile model for semantic similarity, capable of capturing deep contextual relationships. However, its effectiveness in applications like ASAG depends on factors such as training strategy, dataset type, and evaluation setup, making comparative analysis with other architectures essential.

2.2 Summary Table

Table 6 – Summary of Embeddings and Transformers used in ASAG

Model	Embedding	Transformer	Advantages	Limitation
Traditional ASAG (Rule-based) [1]	None (keyword matching)	Not used	Simple, fast, easy to implement	Cannot capture semantic meaning, fails for paraphrased answers
Word2Vec / GloVe [1]	Word Embeddings	Not used	Computationally efficient, good baseline performance	No context awareness, same word has same meaning, poor for phrases
BERT [1]	Contextual Embeddings	Encoder based transformer	Captures context, strong semantic understanding	Computationally expensive, resource intensive
Sentence-BERT (SBERT) [2]	Sentence Embeddings	SBERT	Efficient similarity comparison, ideal for semantic tasks like ASAG	Still requires significant computation
Pythia [3]	Contextual Embeddings	Autoregressive Transformer	Helps analyze model learning, strong contextual understanding	Complex, not specifically optimized for grading tasks
T5 [4]	Sentence Embeddings (text-to-text)	Encoder – Decoder based transformer	Flexible, powerful, captures deep semantic relations	High computational cost, requires careful fine-tuning

2.3 Limitations of Existing Methods

Early approaches to automated assessment, particularly lexical and rule-based methods, suffer from poor semantic understanding. These techniques rely heavily on keyword matching or simple concept overlap, which makes them ineffective when students express correct answers using different wording. Similarly, word embedding models such as Word2Vec and GloVe lack contextual awareness, as they assign a single vector representation to each word regardless of its meaning in different contexts.

More advanced techniques, including contextual embeddings like BERT, address some of these issues by incorporating context into word representations. However, they come with a high computational cost, requiring substantial resources for training and fine-tuning, which limits their practicality in many applications. Sentence embeddings attempt to capture the overall meaning of text, but their effectiveness is inconsistent, and it remains unclear whether they consistently outperform simpler approaches across different datasets.

Similarity-based grading methods introduce additional limitations, particularly due to their reliance on linear score mappings. Converting cosine similarity directly into grades assumes a linear relationship that does not always align with human grading patterns. Moreover, these approaches lack robust feature learning, as they depend solely on similarity scores and fail to capture richer interactions between representations that could enhance predictive accuracy.

Another significant challenge lies in model generalization and handling diverse student responses. Performance often varies across datasets such as Mohler, SciEntsBank, and Beetle, indicating limited generalizability. Models also struggle with paraphrased answers or those requiring deeper reasoning. While supervised approaches can improve performance, they depend on large amounts of labeled data, which is expensive to produce. Additionally, many machine learning and transformer-based models function as black boxes, offering limited interpretability and reducing trust. This creates a broader trade-off between scalability and accuracy, where highly accurate models tend to be resource-intensive, while more efficient methods often compromise performance.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 Proposed Methodology

To evaluate the effectiveness of transformer-based models for automated short-answer grading, a series of experiments are designed using multiple datasets and model architectures. The experiments aim to analyze how different models perform in understanding semantic similarity between student answers and reference answers. The experimental design, as shown in figure 1, consists of three main experiments, each focusing on a different approach to automated grading. These experiments allow comparison between similarity-based methods, feature-based machine learning approach, and fine-tuned models.

3.1.1 System Architecture

a. Data Layer

The data layer manages all datasets used for training and evaluation. Three publicly available short-answer grading datasets are used:

- Mohler Dataset
- SciEntsBank Dataset
- Beetle Dataset

The datasets are stored in the project directory under:



```
Data/  
  raw/  
  processed/
```

The **raw folder** stores the original datasets, while the **processed folder** contains cleaned and standardized versions used for experiments.

b. Preprocessing Module

The preprocessing module prepares datasets for modeling. It performs several steps including:

- Text normalization and cleaning
- Column standardization across datasets
- Score mapping and formatting
- Dataset structuring into a unified format

A shared utility file:

```
text_cleaning.py
```

handles generic text cleaning operations such as removing unnecessary whitespace and standardizing text formatting.

The final processed datasets contain the standardized fields:

```
reference_answer  
student_answer  
score
```

c. Embedding Generation Module

The embedding module generates **semantic vector representations** for both reference answers and student answers. These embeddings capture the semantic meaning of the text and enable similarity-based grading.

Three transformer-based models are used:

Table 6- Embedding Generation Model

Model	Architecture	Implementation File
SBERT (all-mpnet-base-v2)	Encoder	Sbert_embedder.py
Pythia (pythia-160m)	Decoder	Pythia_embedder.py
T5 (t5-base)	Encoder-Decoder	T5_embedder.py

These modules are in:

```
src/embeddings/
```

Each embedder converts input text into a numerical embedding vector using the HuggingFace Transformers framework.

d. Experiment Module

The experiment module controls the execution of different experimental setups designed to evaluate the grading system. Three experiments are designed:

Table 7- Experiment Modules

Experiment	Purpose
Experiment 1	Embedding similarity based baseline

Experiment 2	Regression without Embedder model fine tuning
Experiment 3	Fine-tuning Tranformer models

The experiments are implemented in:

```
src/experiments/
  experiment1_similarity.py
  experiment2_zeroshot.py
  experiment3_finetune.py
```

e. Evaluation Module

The evaluation module measures how well the predicted similarity scores align with human grading. Multiple metrics are used to provide a comprehensive evaluation:

- Pearson Correlation
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)
- Quadratic Weighted Kappa (QWK)

These metrics are implemented in:

```
src/evaluation/
  evaluation_metrics.py
  run_evaluation_exp1.py
```

The evaluation pipeline processes the experiment outputs and generates summary results stored in:

```
results/experiment1/
```

f. Results and Visualization

Experimental outputs are stored in the **results directory**, which contains similarity results, evaluation summaries, and visualizations.

```
results/
  experiment1/
    metrics_summary.csv
    pearson_comparison.png
    rmse_comparison.png
    pearson_heatmap.png
```

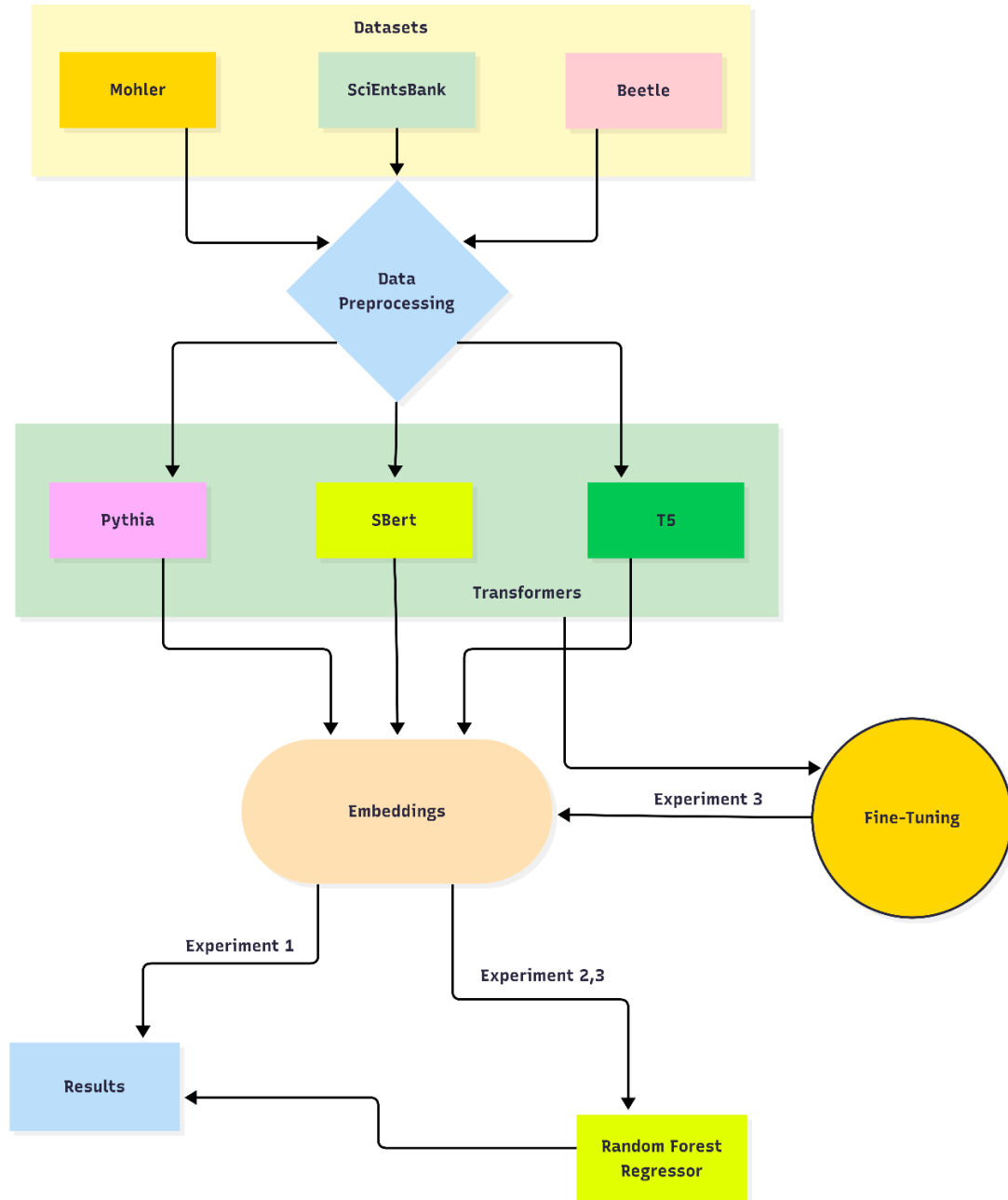


Figure 2: Overall pipeline of our system

3.1.2 Transformer Models Used -

A) SBERT (Sentence-BERT)

SBERT is an encoder-based transformer model designed specifically for generating sentence embeddings. It converts input sentences into fixed-length vector representations that capture semantic meaning. In this project, the all-mpnet-base-v2 model is used to generate embeddings for both reference answers and student answers. These embeddings are compared using cosine similarity to measure how closely a student's answer matches the reference answer.

B) Pythia

Pythia is a decoder-only transformer language model developed by EleutherAI. Unlike encoder models, decoder models are trained using a causal language modeling objective, where the model predicts the next token in a sequence. In this project, the pythia-160m variant is used. Although the model is primarily designed for language generation tasks, the hidden states generated by the transformer layers can also be used to obtain semantic representations of text. To generate embeddings, the hidden states corresponding to each token in the sentence are extracted and aggregated to produce a single vector representation. This vector is then used to compute similarity between student and reference answers. Pythia is included in the system to evaluate how well a general-purpose language model performs on semantic similarity tasks compared to models specifically trained for sentence embeddings.

C) T5 (text-to-text transfer transformer)

T5 is an encoder-decoder transformer architecture designed to convert all natural language processing tasks into a unified text-to-text format. In this architecture, the encoder processes the input text and produces contextual representations, while the decoder generates output sequences. In this project, the t5-base model is used. The encoder component of the model is used to generate embeddings for reference and student answers. These embeddings are obtained from the encoder's hidden states and pooled into a single sentence representation. T5 is widely used for tasks such as text summarization, machine translation, and question answering. Including T5 in the system allows evaluation of how encoder-decoder models perform in semantic similarity-based grading tasks.

The use of three different transformer architectures enables comparative analysis of their performance in automated short-answer grading. The models represent three major categories of transformer architectures:

Model	Architecture Type
SBERT	Encoder
Pythia	Decoder
T5	Encoder-Decoder

By comparing these models, the system can analyze how architecture type affects the quality of semantic representations and grading performance.

3.2. Tool and Technique Selection

3.2.1 Tool Selection

- **Programming Language: Python**
- **Libraries and Frameworks**

1. **HuggingFace Transformers:** The HuggingFace Transformers library was used to load and utilize pre-trained transformer models such as Pythia and T5. It provides APIs for accessing model architectures, tokenizers, and hidden representations required for generating text embeddings.
2. **Sentence Transformers:** The Sentence Transformers library was used to implement SBERT-based embeddings. It provides efficient utilities for generating sentence embeddings and computing semantic similarity between sentences.
3. **PyTorch:** PyTorch was used as the underlying deep learning framework for running transformer models and generating embeddings. It provides tensor operations and GPU acceleration for neural network computations.
4. **Scikit-learn:** Scikit-learn was used for computing evaluation metrics such as RMSE, MAE, and Quadratic Weighted Kappa. It also provides utilities for similarity computation and model evaluation.
5. **Pandas:** Pandas was used for dataset manipulation, preprocessing, and storage of experimental results.
6. **Matplotlib and Seaborn:** These libraries were used to generate visualization plots for analyzing experimental results, including performance comparisons across models and datasets.

To run transformer-based models efficiently, the following hardware configuration is recommended:

- **Processor:** Intel i5 / equivalent or higher
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** Minimum 10–20 GB for datasets and model files
- **GPU (optional but recommended):** NVIDIA GPU with CUDA support for faster model training and fine-tuning .

3.2.2 Dataset Selection

For the automated short-answer grading system, three publicly available datasets were selected that are widely used in research on automatic grading and semantic similarity.

The datasets used in this project are: **Mohler Dataset**, **SciEntsBank Dataset** and **Beetle Dataset**.

These datasets were chosen because they provide:

- Human-graded short answers
- Reference answers for each question
- Multiple grading levels
- Diverse student responses

Using multiple datasets allows evaluation of the system across **different domains and answer styles**, improving the robustness of the experimental analysis.

A. Mohler Dataset

The **Mohler dataset** is a commonly used dataset for short-answer grading research. It contains student responses to questions from computer science courses. Each record in the dataset includes:

Question	Reference (model) answer	Student answer	Human grading scores
----------	--------------------------	----------------	----------------------

Multiple human graders evaluate the student's responses, and the final score used in this project is the **average score assigned by the evaluators**.

Score range: 0 – 5, where 0 indicates incorrect answers and 5 indicates completely correct answers.

B. SciEntsBank Dataset

The **SciEntsBank dataset** is part of the Semantic Evaluation Workshop Task on student response analysis and contains answers to science-related questions. Each record includes:

Question	Reference (model) answer	Student answer	Human grading scores
----------	--------------------------	----------------	----------------------

The dataset categorizes student answers into five classes:

LABEL	0	1	2	3	4
MEANING	Non- domain	Irrelevant	Partially correct	Contradictory	Correct

C. Beetle Dataset

The **Beetle dataset** is another dataset from the Semantic Evaluation Workshop student response analysis task. It contains short answers to technical and conceptual questions. Each entry includes:

Question	Reference (model) answer	Student answer	Human grading scores
----------	--------------------------	----------------	----------------------

The scoring scheme is like SciEntsBank and includes the following categories:

LABEL	0	1	2	3	4
MEANING	Irrelevant	Non-domain	Partially correct	Contradictory	Correct

This dataset helps evaluate the system's ability to grade answers in technical domains.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Experiment Setup

4.1.1 Experiment 1: Embedding Similarity Baseline

The first experiment establishes a **baseline approach** using semantic similarity between student answers and reference answers.

In this experiment, both the reference answer and the student answer are converted into vector embeddings using transformer models. The similarity between these embeddings is then calculated using cosine similarity.

The similarity score is treated as an indicator of how closely the student answer matches the reference answer.

The steps involved in this experiment as illustrated in the figure 2, are:

- **Load Datasets-** The processed Mohler, SciEntsBank, and Beetle datasets are loaded from the processed data directory using the Pandas library. Each dataset contains standardized fields: reference_answer, student_answer, and score.
- **Embedding Generation-** Embeddings for reference answers and student answers are generated using transformer models. The models evaluated include SBERT, Pythia, and T5, which convert text into numerical embeddings to measure semantic similarity between responses.
- **Similarity Computation** - Once embeddings are generated for both reference and student answers, the similarity between them is calculated using cosine similarity. Cosine similarity measures the angle between two vectors and provides a score indicating how similar the two embeddings are. A higher similarity score indicates a stronger semantic match between the student's answer and the reference answer. For each pair of answers, a similarity value is computed and stored alongside the original dataset.
- **Convert similarity scores into predicted grading scores** -The cosine similarity scores, which range from -1 to 1 , were first normalized to a $0-1$ scale using a linear transformation. This was done by adding 1 to each similarity value and dividing by 2 . The normalized values were then scaled to match the grading range of the dataset by multiplying with the difference between the maximum and minimum possible scores and adding the minimum score. This process results in predicted grading scores
- **Evaluation-** The similarity scores generated in the experiment are evaluated by comparing them with human-assigned grading scores. To analyze model performance, visualization plots were generated from the evaluation results. The following plots were generated:

- a. Pearson correlation comparison across models and datasets
- b. RMSE comparison for prediction error analysis
- c. MAE comparison for average prediction error analysis
- d. QWK comparison for agreement between predicted scores and human grading
- e. Heatmap visualization showing model performance across datasets

4.1.2 Experiment 2: Regression without Embedder model Fine-Tuning

The second experiment extends the baseline similarity approach by incorporating feature engineering and a machine learning model for score prediction. Instead of directly using cosine similarity as the final score, multiple features are derived from the embeddings of reference answers and student answers.

In this approach, embeddings for both reference and student answers are first generated using transformer-based models. These embeddings are then used to construct a feature vector representing the relationship between the two answers.

The constructed feature vectors are then used to train a **Random Forest Regressor**, which predicts the grading score based on these features.

The steps involved in this experiment as illustrated in the figure 3, are:

- **Embedding Retrieval** – The embeddings generated in step 2 of experiment 1 are retrieved to continue with our experiment 2.
- **Feature Engineering** - Instead of directly using similarity scores, multiple features are constructed from the embeddings of reference and student answers. These features capture both semantic similarity and vector-level relationships. The following features are computed for each pair of answers:

- a. Cosine similarity between embeddings
- b. Absolute difference between embedding vectors
- c. Element-wise product of embedding vectors

These features are concatenated to form a feature vector representing the relationship between the student answer and the reference answer.

- **Model Training** - The constructed feature vectors are used to train a Random Forest Regressor. The model learns the relationship between the engineered features and the corresponding grading scores. Random Forest is selected due to its ability to handle high-dimensional feature spaces and capture non-linear relationships between variables. The model is trained using the training portion of the dataset.

- **Prediction & Result Storage** - After training, the model is used to predict grading scores for the test dataset. The predicted scores are stored along with the original dataset entries.
- **Evaluation of Predictions** - The predicted scores generated by the model are evaluated by comparing them with human-assigned scores. The evaluation process follows the same framework as Experiment 1 to ensure consistency.
- **Visualization of Experimental Results** - Visualization plots are generated to analyze and compare model performance across datasets and embedding models. The following plots were generated:
 - a. Pearson correlation comparison across models and datasets
 - b. RMSE comparison for prediction error analysis
 - c. MAE comparison for average prediction error analysis
 - d. QWK comparison for agreement between predicted scores and human grading
 - e. Heatmap visualization showing model performance across datasets

4.1.3 Experiment 3: Fine-Tuned Grading Model

The third experiment focuses on fine-tuning transformer models on the grading datasets to improve performance.

In this approach, the models are trained using reference answers, student answers, and corresponding human-assigned scores. The training process allows the model to learn patterns specific to the grading task and generate improved semantic representations.

The steps involved in this experiment as illustrated in the figure 4, are:

- **Transformer fine tuning**- The transformer models are fine-tuned on the training portion of the dataset using an 80:20 train-test split. During training, the models learn to encode relationships between reference answers and student answers based on their associated scores. The fine-tuning process is performed for a fixed number of epochs (5 epochs in this implementation), ensuring a balance between learning efficiency and computational cost. The trained models are saved for reuse in subsequent steps.
- **Embedding Generation using Fine-Tuned Models** - After fine-tuning, the trained transformer models are used to generate embeddings for both reference answers and student answers. These embeddings are expected to be more aligned with the grading task compared to pre-trained embeddings, as the models have learned dataset-specific patterns during training.
- **Model Training using Embeddings** - The embeddings generated from the fine-tuned models are used as input features for a Random Forest Regressor. The regression model is trained to learn the

mapping between embeddings and grading scores. This step combines the representational power of transformer models with the predictive capability of a machine learning model.

- **Prediction & Result Storage** - The trained Random Forest model is used to predict grading scores for the test dataset. The predicted scores are stored along with the corresponding dataset entries.
- **Results and visualization** – Same evaluation metrics as experiment 1 and 2 are kept for this experiment as well. Graphs for the same have been plotted for result analysis.

Fine-tuning allows the model to adapt its representations specifically for the automated grading task, which improves the quality of embeddings and enhances prediction accuracy.

4.2 Flow Charts and Test Metrics

4.2.1 Flow Charts

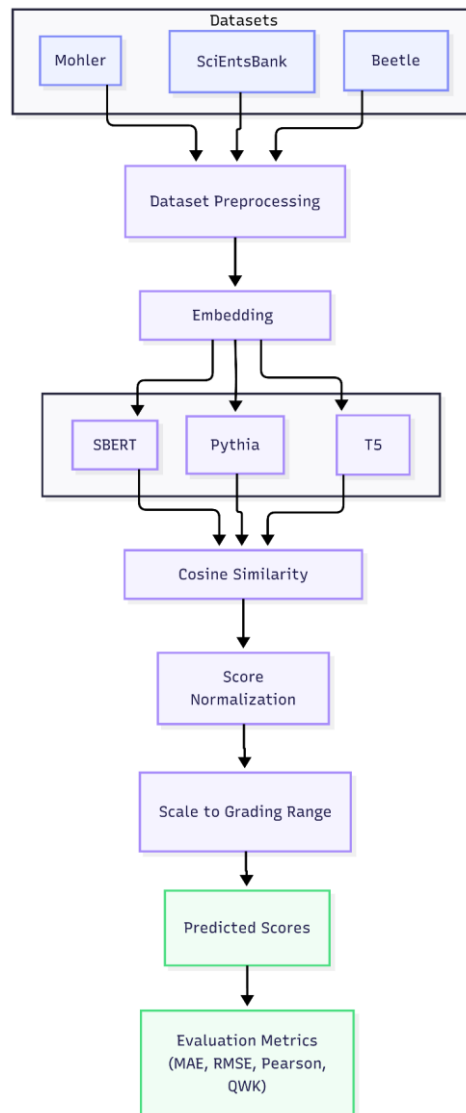


Figure 3-System architecture diagram for running transformer-based STS task.

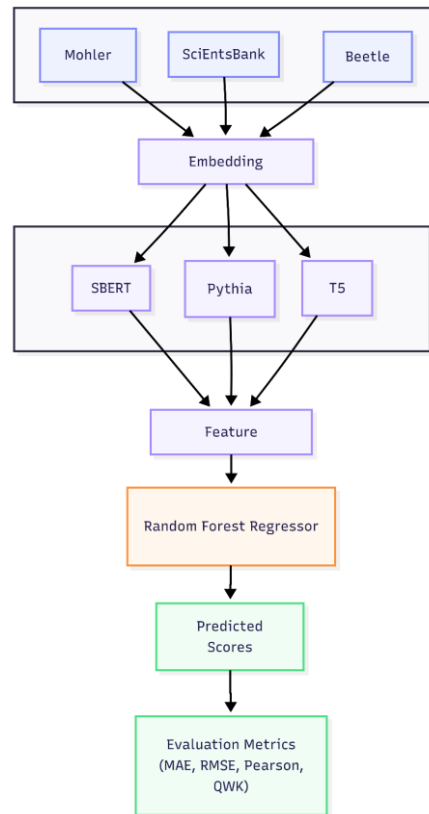


Figure 4- Work Flow for model without task specific training

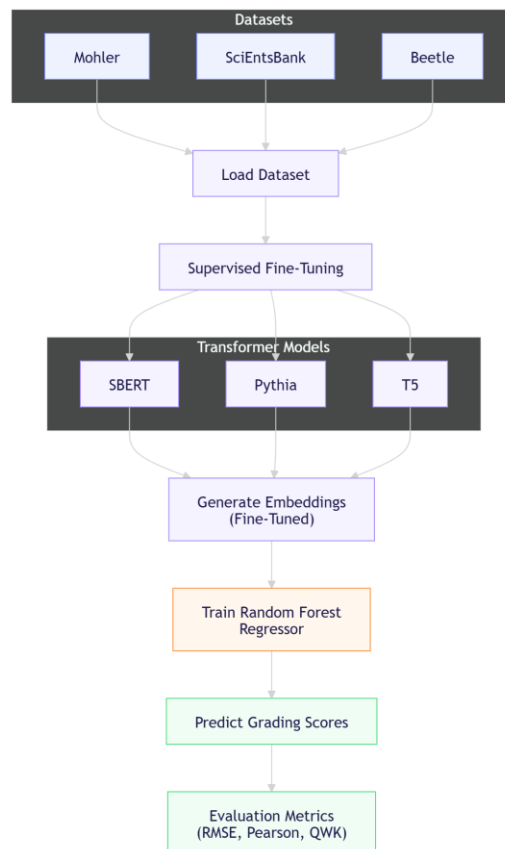


Figure 5: Work Flow of model with task specific training

4.2.2 Evaluation Metrics

A) Pearson Correlation

Pearson Correlation measures the **linear relationship between predicted scores and human-assigned scores**. It evaluates how well the model predictions follow the same trend as human grading. The Pearson correlation coefficient ranges from **-1 to 1**, where: 1 means best results while -1, worst. In the context of automated grading, a higher Pearson correlation indicates that the model is able to rank student answers in a way that closely matches human evaluators. Pearson correlation is commonly used in semantic similarity tasks because it captures how well similarity scores align with human judgments.

B) Root Mean Squared Error (RMSE)

Root Mean Squared Error measures the **difference between predicted scores and actual human scores**. It calculates the square root of the average squared differences between predicted and true scores. RMSE is useful because it penalizes large prediction errors more strongly than smaller ones. A lower RMSE value indicates that the model's predictions are closer to the human-assigned scores. This metric helps evaluate how accurately the automated system predicts grading scores.

C) Mean Absolute Error (MAE)

Mean Absolute Error measures the **average absolute difference between predicted scores and actual scores**. Unlike RMSE, MAE does not square the errors, which means all errors contribute equally to the final metric. MAE provides an easily interpretable measure of prediction accuracy. A lower MAE value indicates that the model predictions are closer to the true scores assigned by human evaluators. Using both RMSE and MAE allows a more complete understanding of prediction errors.

C) Quadratic Weighted Kappa (QWK)

Quadratic Weighted Kappa measures the **level of agreement between predicted scores and human scores** while considering the ordinal nature of grading scales. Unlike simple accuracy measures, QWK accounts for the severity of disagreement between predicted and actual scores. Larger differences between grades are penalized more heavily than smaller differences.

The QWK score ranges from **0 to 1**, where:

- **1** indicates perfect agreement
- **0** indicates no agreement beyond chance

Quadratic Weighted Kappa is widely used in automated essay scoring and grading systems because it reflects how closely the model grading aligns with human evaluators

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Result analysis

5.1.1 Experiment 1: Similarity Based Baseline

This experiment evaluates the performance of transformer models using **cosine similarity between reference and student answer embeddings**.

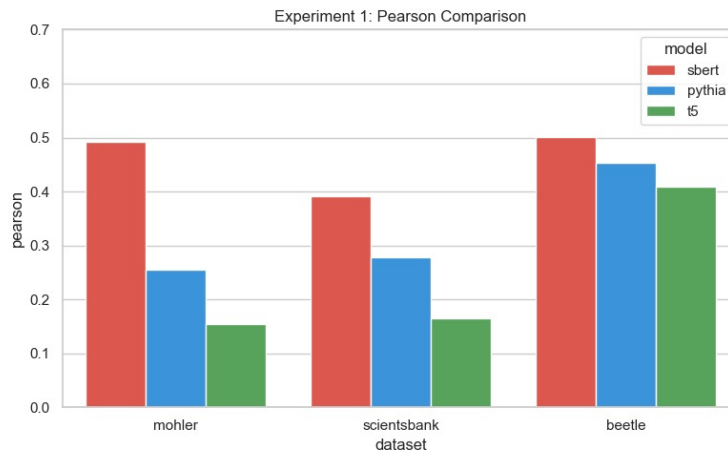


Figure 6: Pearson Correlation Comparison for Experiment 1

The Pearson correlation values indicate that SBERT achieves the highest performance across all datasets, with values of 0.49 (Mohler), 0.39 (SciEntsBank), and 0.50 (Beetle). Pythia shows moderate performance, while T5 performs the weakest.

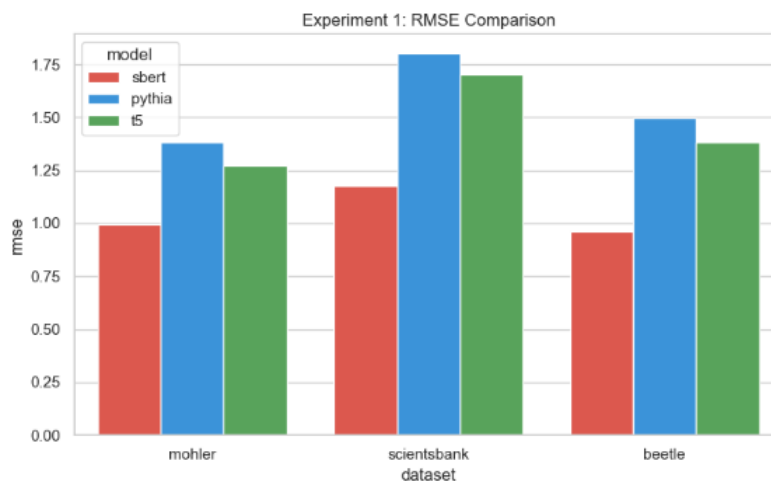


Figure 7: RMSE Comparison for Experiment 1

The RMSE values are highest for T5 and Pythia, indicating larger prediction errors, whereas SBERT shows relatively lower error values.

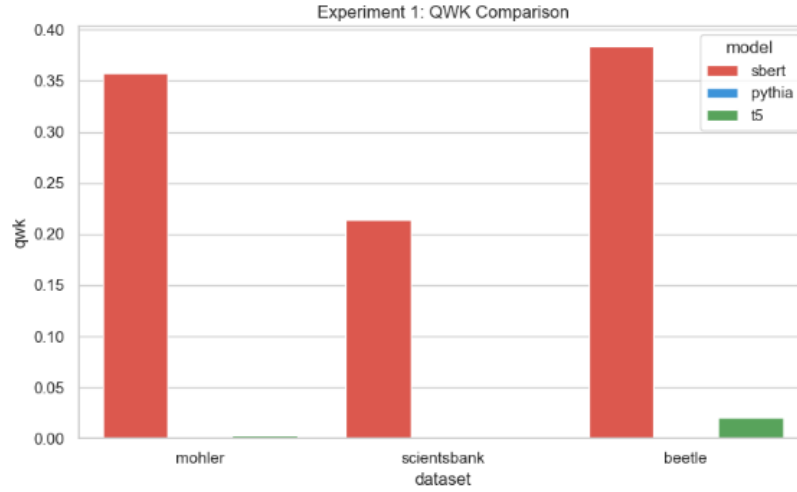


Figure 8: QWK Comparison for Experiment 1

Quadratic Weighted Kappa (QWK) values are close to zero for Pythia and T5, indicating poor agreement with human grading. SBERT achieves moderate agreement.

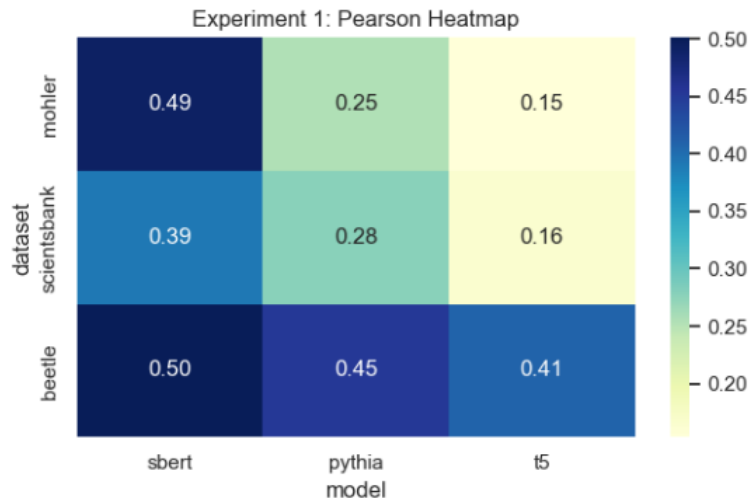


Figure 9: Pearson Heatmap for Experiment 1

The heatmap clearly shows that SBERT consistently outperforms other models across all datasets. This experiment demonstrates that **semantic similarity alone is insufficient** for accurate grading, as it cannot capture nuanced scoring patterns.

5.1.2 Experiment 2: Regression without Embedder model Fine-Tuning

This experiment introduces a Random Forest regressor trained on frozen transformer embeddings.

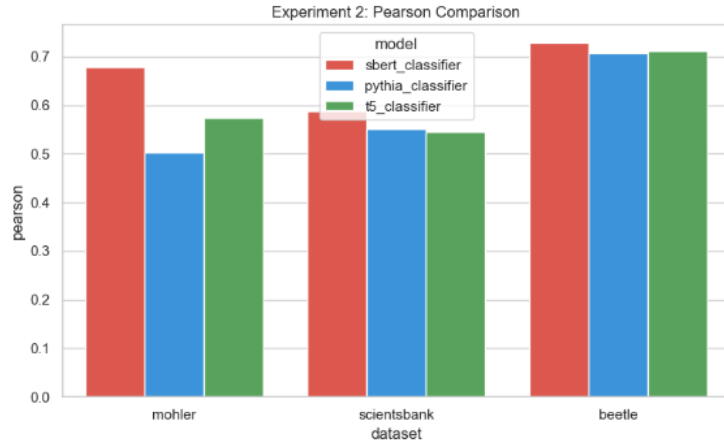


Figure 10: Pearson Comparison for Experiment 2

A significant improvement is observed in Pearson correlation across all datasets. SBERT achieves values up to 0.72 (Beetle), demonstrating the effectiveness of supervised learning.

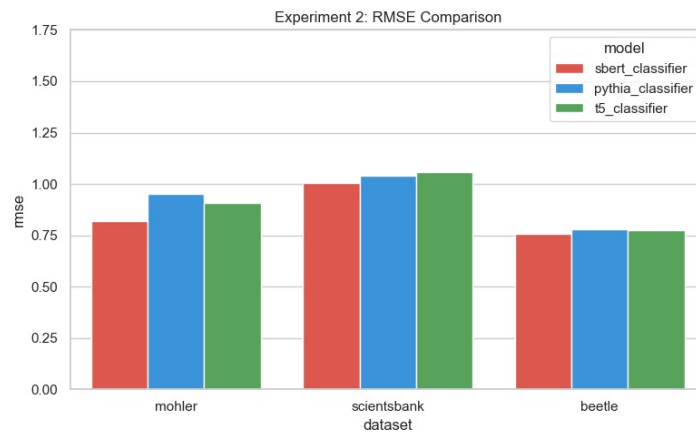


Figure 11: RMSE Comparison for Experiment 2

The RMSE values decrease compared to Experiment 1, indicating improved prediction accuracy.

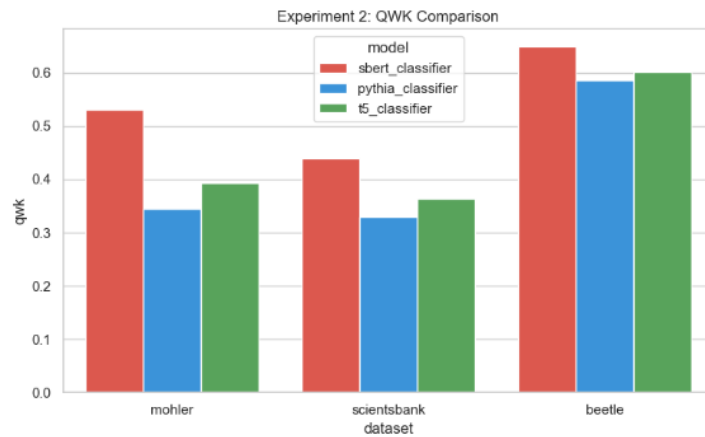


Figure 12: QWK Comparison for Experiment 2

QWK values improve significantly, reaching up to 0.64, indicating better agreement with human graders.

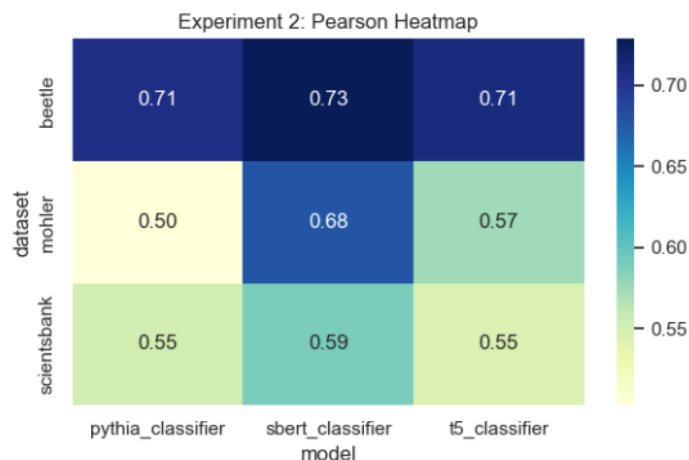


Figure 13: Pearson Heatmap for Experiment 2

The heatmap highlights consistent improvement across all models, especially for SBERT and T5.

The introduction of a classifier allows the model to **learn the mapping between embeddings and scores**, significantly improving performance.

5.1.3 Experiment 3: Fine-Tuning Transformer Models

This experiment fine-tunes transformer models directly on the grading task.

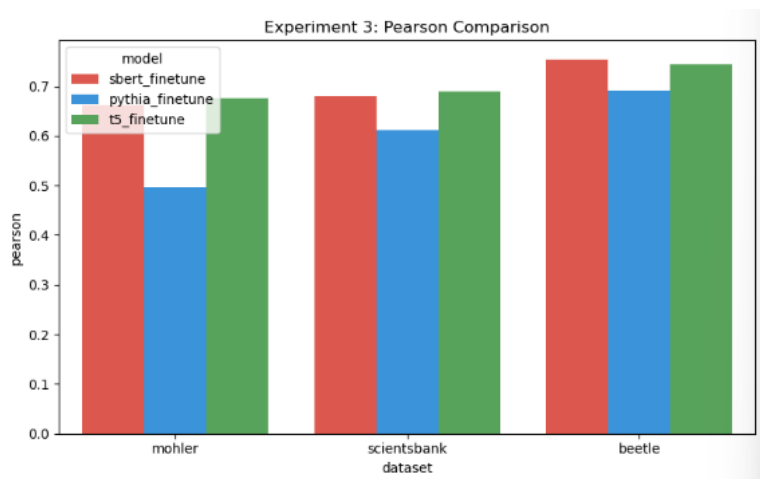


Figure 14: Pearson Comparison for Experiment 3

Fine-tuned models achieve the highest Pearson correlation values, with T5 and SBERT reaching up to 0.75.

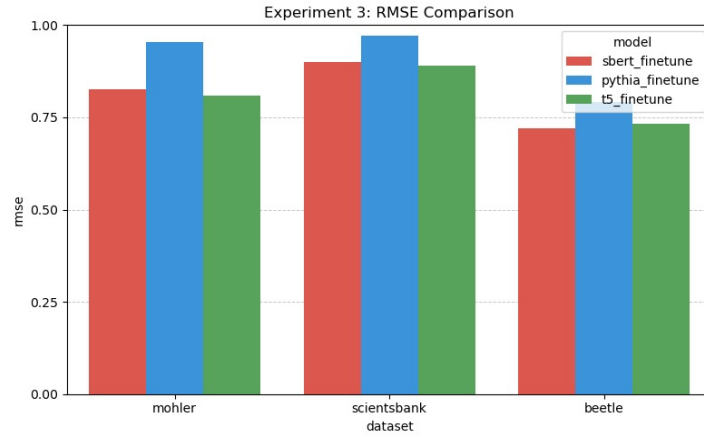


Figure 15: RMSE Comparison for Experiment 3

RMSE values are lowest in this experiment, indicating improved prediction accuracy.

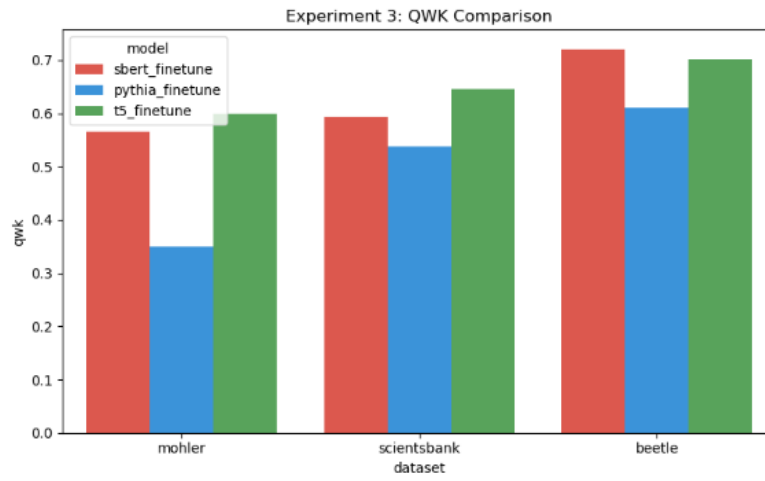


Figure 16: QWK Comparison for Experiment 3

QWK values reach up to 0.72, showing strong agreement with human evaluation.

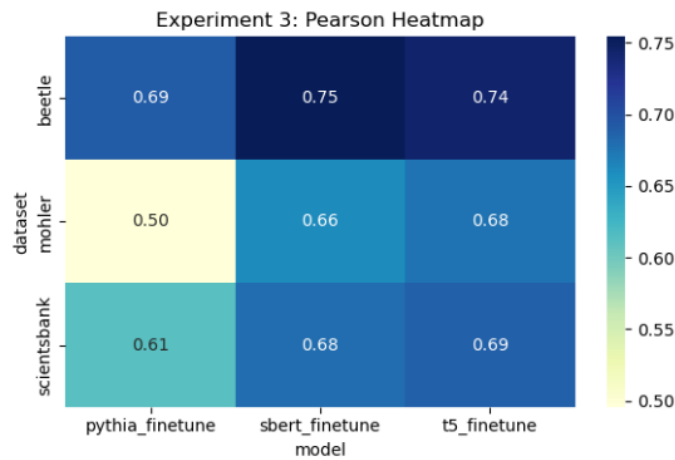


Figure 17: Pearson Heatmap for Experiment 3

The heatmap confirms that fine-tuning consistently yields superior performance across all datasets. Fine-tuning enables the model to **learn task-specific representations**, resulting in the best performance.

5.2 Performance Evaluation

5.2.1 Comparison Based on Pearson Correlation

Pearson correlation is the primary metric used to measure alignment between predicted scores and human grading.

- **Experiment 1:**
 - Maximum Pearson ≈ 0.50 (SBERT, Beetle)
 - Indicates moderate alignment with human evaluation
- **Experiment 2:**
 - Improves to ≈ 0.72 (SBERT, Beetle)
 - Shows that supervised learning captures grading patterns
- **Experiment 3:**
 - Further improves to ≈ 0.75 (SBERT, Beetle) and 0.68 (T5, SciEntsBank)
 - Demonstrates strong alignment with human grading

Interpretation:

The improvement from Experiment 1 to Experiment 2 highlights that: **Similarity alone is insufficient; learning a mapping improves performance.**

The further improvement in Experiment 3 shows that: **Task-specific fine-tuning enables the model to better understand grading semantics.**

5.2.2 Error Analysis using RMSE and MAE

RMSE and MAE measure prediction error.

- **Experiment 1:**
 - Highest error across all datasets
 - Example: SciEntsBank RMSE ≈ 1.80 (Pythia)
- **Experiment 2:**
 - Significant reduction in error
 - Example: RMSE reduced to ≈ 1.00 (SBERT)

- **Experiment 3:**
 - Lowest error values
 - Example: RMSE $\approx$ **0.89 (T5, SciEntsBank)**

Interpretation: The decrease in RMSE and MAE across experiments indicates that: Model predictions become closer to actual human scores as learning improves.

5.2.3 Agreement with Human Grading (QWK Analysis)

Quadratic Weighted Kappa (QWK) measures agreement between predicted and actual scores.

- **Experiment 1:**
 - QWK $\approx$ **0 for Pythia and T5**
 - Indicates poor agreement
- **Experiment 2:**
 - QWK improves to **0.3–0.6 range**
 - Shows moderate agreement
- **Experiment 3:**
 - QWK reaches **0.72 (SBERT, Beetle)** and **0.64 (T5, SciEntsBank)**
 - Indicates strong agreement with human grading

Interpretation: Fine-tuned models closely replicate human scoring behavior.

5.2.4 Model-wise Comparison

Table 6: Comparison of transformer models performance across the three experiments

Parameters	Experiments	Worst	Medium	Best
Pearson Correlation	Exp 1	T5 (0.15)	Pythia (0.25)	SBERT (0.49)
	Exp 2	Pythia (0.50)	T5 (0.57)	SBERT (0.67)
	Exp 3	Pythia (0.49)	SBERT (0.66)	T5 (0.67)
QWK	Exp 1	Pythia (0)	T5(0.00347)	SBERT (0.35)
	Exp 2	Pythia (0.34)	T5 (0.39)	SBERT (0.53)
	Exp 3	Pythia (0.34)	SBERT (0.56)	T5 (0.59)
RMSE	Exp 1	Pythia (1.38)	T5 (1.27)	SBERT (0.99)

MAE	Exp 2	Pythia (0.95)	T5 (0.90)	SBERT (0.81)
	Exp 3	Pythia (0.95)	SBERT (0.82)	T5(0.80)
	Exp 1	Pythia (0.83)	T5(0.818)	SBERT (0.811)
	Exp 2	Pythia (0.74)	T5(0.70)	SBERT (0.61)
	Exp 3	Pythia (0.75)	SBERT (0.59)	T5 (0.56)

*As per Mohler dataset.

Key Insight:

Models designed for similarity (SBERT) perform best initially, while generative models (T5) benefit most from fine-tuning.

5.2.5 Dataset-wise Analysis

Table 7: Comparison of datasets performance across the three experiments

Parameters	Experiments	Worst	Medium	Best
Pearson Correlation	Exp 1	SciEntsBank (0.39)	Mohler(0.49)	Bettle (0.50)
	Exp 2	SciEntsBank (0.58)	Mohler (0.67)	Bettle (0.72)
	Exp 3	Mohler (0.66)	SciEntsBank (0.67)	Bettle (0.75)
QWK	Exp 1	SciEntsBank (0.21)	Mohler(0.35)	Bettle (0.38)
	Exp 2	SciEntsBank (0.43)	Mohler (0.53)	Bettle (0.64)
	Exp 3	Mohler (0.56)	SciEntsBank (0.59)	Bettle (0.72)
RMSE	Exp 1	SciEntsBank (1.17)	Mohler(0.99)	Bettle (0.95)
	Exp 2	SciEntsBank (1.00)	Mohler (0.81)	Bettle (0.75)
	Exp 3	SciEntsBank (0.90)	Mohler (0.82)	Bettle (0.71)
MAE	Exp 1	SciEntsBank (1.03)	Mohler(0.81)	Bettle (0.77)
	Exp 2	SciEntsBank (0.86)	Mohler (0.61)	Bettle (0.59)
	Exp 3	SciEntsBank (0.71)	Mohler (0.59)	Bettle (0.54)

*As per SBERT transformer.

KEY INSIGHTS;

Why was SBERT initially the best performing model?

SBERT achieved the best performance in the baseline experiment because it is specifically trained for **sentence embedding and semantic similarity tasks**. Its training objective aligns closely with the grading approach used in Experiment 1, where cosine similarity between embeddings is used to estimate scores. As a result, SBERT produces more meaningful and comparable sentence representations, leading to higher correlation with human grading.

Why was Pythia the worst performing model?

Pythia performed poorly in the baseline because it is a **decoder-only language model trained for next-token prediction**, not for generating sentence-level embeddings. The embeddings extracted from such models are not optimized for semantic similarity, resulting in weak alignment between similarity scores and actual grades. Consequently, it shows low Pearson correlation and near-zero agreement (QWK) in Experiment 1.

Why did T5 perform better after fine-tuning?

T5 showed significant improvement after fine-tuning because it is a **sequence-to-sequence model designed to learn task-specific mappings**. While its raw embeddings are not suitable for similarity comparison, fine-tuning allows the model to directly learn the relationship between input text (question, reference, student answer) and the target score. This enables T5 to adapt effectively to the grading task, resulting in improved performance.

Why did the SciEntsBank dataset perform the worst, while others performed better?

The SciEntsBank dataset is more challenging due to its **high variability in student responses**, including diverse phrasing, partial correctness, and domain-specific language. This makes it difficult for models to capture consistent semantic patterns, especially in baseline approaches. In contrast, datasets like Beetle contain shorter and more structured answers, making semantic similarity easier to detect, which leads to better performance across models.

How did regression and fine-tuning improve the results?

The improvement in performance from Experiment 1 to Experiment 2 and Experiment 3 can be attributed to the increasing ability of the system to learn the relationship between textual representations and grading scores.

Role of Regression (Experiment 2)

In Experiment 1, grading was based solely on **cosine similarity between embeddings**, which assumes that higher semantic similarity directly corresponds to higher scores. However, this assumption is often insufficient because grading depends on multiple nuanced factors such as partial correctness, missing details, or contradictory information.

In Experiment 2, a regression model (Random Forest) was introduced on top of frozen transformer embeddings. This significantly improved performance because:

- The regression model learns a **non-linear mapping** between embedding features and actual scores.
- It utilizes richer feature representations such as cosine similarity, element-wise differences, and interactions between embeddings.
- It captures complex grading patterns that cannot be modeled by simple similarity measures.

As a result, the model produces predictions that are **closer to human-assigned scores**, leading to improved Pearson correlation and reduced RMSE and MAE values.

Role of Fine-Tuning (Experiment 3)

While regression improves performance using fixed embeddings, the embeddings themselves are still **generic and not optimized for grading tasks**. Fine-tuning addresses this limitation.

In Experiment 3, transformer models were fine-tuned directly on the grading dataset, allowing the model to:

- Learn **task-specific representations** tailored for evaluating student answers.
- Adjust internal weights to focus on relevant aspects such as correctness, completeness, and semantic alignment.
- Capture deeper contextual relationships between question, reference answer, and student response.

Unlike Experiment 2, where only the final mapping is learned, fine-tuning enables **end-to-end learning**, where both representation and prediction layers are optimized simultaneously.

This results in:

- Higher Pearson correlation (better alignment with human grading)
- Lower RMSE and MAE (reduced prediction error)
- Higher QWK (stronger agreement with human evaluators)

Final Comparison:

Traditional Methods < Similarity < Classifier < Fine-tuned Transformer

Chapter 6: CONCLUSION AND FUTURE SCOPE

6.1 Summary of Work and Achievements

This project presents an automated system for short answer grading using transformer-based models. The work was structured into three progressive experimental stages to evaluate different approaches for scoring student answers.

Initially, a similarity-based baseline was developed using transformer-generated embeddings and cosine similarity to estimate scores. This provided a simple yet effective starting point but demonstrated limited alignment with human grading.

Subsequently, a supervised learning approach was introduced, where a machine learning classifier was trained on top of frozen transformer embeddings. This significantly improved performance by learning the relationship between semantic representations and grading patterns.

Finally, transformer models were fine-tuned directly on the grading task, enabling the system to learn task-specific representations. This approach achieved the highest performance across all datasets, demonstrating strong agreement with human evaluators.

The system was evaluated on three benchmark datasets: Mohler, SciEntsBank, and Beetle, using multiple evaluation metrics including Pearson correlation, RMSE, MAE, and Quadratic Weighted Kappa (QWK). The results consistently showed that fine-tuned models outperform both similarity-based and classifier-based approaches.

In addition to implementation, the project includes:

- A modular pipeline for preprocessing, embedding generation, training, and evaluation
- Comparative analysis across multiple transformer architectures (SBERT, Pythia, T5)
- Visualization of results using bar charts and heatmaps

This structured approach provides a comprehensive understanding of how different modeling strategies impact automated grading performance.

6.2 Impact on Society, Environmental Sustainability, and Ethical Considerations

Impact on Society

Automated grading systems can significantly enhance the education system by:

- Reducing manual workload for educators
- Providing faster and more consistent feedback to students
- Enabling scalable assessment in large classrooms and online learning platforms

Such systems can support personalized learning by allowing students to receive immediate feedback and improve their understanding.

Industry Relevance

The proposed system has strong relevance in:

- EdTech platforms (e.g., online courses, MOOCs)
- Automated assessment tools
- AI-driven tutoring systems

It can be integrated into learning management systems (LMS) to improve evaluation efficiency and scalability.

Environmental Sustainability

Automating grading reduces reliance on paper-based evaluations and manual processing, contributing to reduced paper consumption and lower administrative overhead. However, transformer models require computational resources, which may increase energy consumption. Efficient model design and optimization can help mitigate this impact.

Ethical Issues and Compliance

Key ethical considerations include:

- **Bias in grading:** Models may inherit biases from training data, leading to unfair evaluation of certain types of responses.
- **Transparency:** Automated grading systems must provide explainable outputs to ensure trust among users.
- **Data privacy:** Student responses must be handled securely and in compliance with data protection regulations.
- **Fairness:** The system should ensure equal treatment across diverse linguistic and educational backgrounds.

6.3 Limitations of the Work

Despite promising results, the proposed system has certain limitations:

- Model performance is influenced by the quality and diversity of training data.
- Transformer-based models act as black-box systems, making it difficult to explain individual grading decisions.
- The model may not perform equally well on unseen subjects or domains without pretraining.
- Performance may degrade for very long or highly detailed responses.
- Fine-tuning transformer models require significant computational resources.
- The system primarily focuses on semantic correctness and may not fully capture creativity or writing quality.

6.4 Future Scope

The project can be extended in several directions:

- **Incorporation of large language models (LLMs):** Zero-shot and few-shot prompting techniques can be explored for grading without explicit training.
- **Multi-modal grading:** Extending the system to evaluate diagrams, code, or mathematical expressions alongside text.
- **Explainable AI integration:** Providing justification for assigned scores to improve transparency and trust.
- **Domain adaptation:** Training models on subject-specific datasets for improved performance in specialized fields.
- **Real-time deployment:** Integrating the system into web-based platforms for live student feedback.
- **Hybrid approaches:** Combining rule-based methods with deep learning for better interpretability and robustness.
- **Bias mitigation techniques:** Ensuring fairness across different student groups and answer styles.
- **Efficiency optimization:** Using lightweight models or distillation techniques to reduce computational cost.

REFERENCES

- [1] Putnikovic, Marko, and Jelena Jovanovic. "Embeddings for automatic short answer grading: A scoping review." *IEEE Transactions on Learning Technologies* 16.2 (2023): 219-231.
- [2] Reimers, Nils, and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks." Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP). 2019.
- [3] Biderman, Stella, et al. "Pythia: A suite for analyzing large language models across training and scaling." International conference on machine learning. PMLR, 2023..
- [4] Ni, Jianmo, et al. "Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models." Findings of the association for computational linguistics: ACL 2022.
- [5] SemEval, "SemEval-2013 Task 7: Interpretable Semantic Textual Similarity," 2013.
- [6] Allen Institute for AI, "SciEntsBank Dataset for Student Answer Grading," 2013.
- [7] University of North Texas, "Beetle Corpus for Short Answer Grading," 2012.

APPENDICES

Code Snippets

```
def encode(self, texts, batch_size=16):  
    embeddings = self.model.encode(  
        texts,  
        batch_size=batch_size,  
        convert_to_numpy=True,  
        show_progress_bar=False,  
        normalize_embeddings=True  
    )  
  
    return embeddings
```

```
from src.embeddings.pythia_embedder import PythiaEmbedder  
from src.embeddings.sbert_embedder import SBERTEmbedder  
from src.embeddings.t5_embedder import T5Embedder  
  
def load_dataset(path):  
    df = pd.read_csv(path)  
  
    ref_answers = df["reference_answer"].tolist()  
    stu_answers = df["student_answer"].tolist()  
    scores = df["score"].tolist()  
  
    return df, ref_answers, stu_answers, scores
```

```
def compute_similarity(embedder, ref_texts, stu_texts):  
    ref_embeddings = embedder.encode(ref_texts)  
    stu_embeddings = embedder.encode(stu_texts)  
  
    similarities = cosine_similarity(ref_embeddings, stu_embeddings)  
    return similarities.diagonal()
```

```
def train_and_evaluate(X, y):  
    indices = np.arange(len(y))  
  
    X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(  
        X, y, indices, test_size=0.2, random_state=42  
    )  
  
    model = RandomForestRegressor(  
        n_estimators=200,  
        n_jobs=-1,  
        random_state=42  
    )
```

```
palette = {  
    "sbert_finetune": "#F44336",  
    "pythia_finetune": "#2196F3",  
    "t5_finetune": "#4CAF50",  
}  
  
def plot_experiment3():  
    csv_path = os.path.join(OUTPUT_DIR, "metrics_summary.csv")  
  
    if not os.path.exists(csv_path):  
        print(f"✗ {csv_path} not found. Run experiment3 first.")  
        return  
  
    df = pd.read_csv(csv_path)  
    df = df.dropna(subset=["pearson", "rmse"])
```


ANNEXURE

Sustainability Statement

The proposed system supports sustainable digital education by enabling automated and scalable grading, reducing the workload on educators while ensuring fair and consistent evaluation of student responses. It promotes environmentally friendly practices by minimizing the use of paper and physical resources through digital assessment methods. Although transformer models require computational resources, the use of efficient and pre-trained models helps in reducing energy consumption.

Team Roles and Contributions

The project was executed collaboratively with a structured division of responsibilities to ensure coverage of all datasets, models, and experimental stages.

Student 1: Pragya Varshney - 9923103039 (Beetle Dataset & T5 Model)

- Preprocessed the **Beetle dataset**, including:
 - Cleaning and standardizing raw data
 - Extracting relevant fields (reference answer, student answer, score)
 - Formatting the dataset for compatibility with the pipeline
- Implemented **Experiment 1 (Similarity-Based Approach)** using T5:
 - Generated embeddings
 - Computed cosine similarity
 - Evaluated results using Pearson, RMSE, MAE, and QWK
- Implemented **Experiment 2 (Classifier without Fine-Tuning)**:
 - Created feature vectors (cosine similarity, difference, product)
 - Applied regression model (Random Forest)
 - Evaluated prediction performance
- Implemented **Experiment 3 (Fine-Tuning)**:
 - Fine-tuned the T5 model on the grading dataset
 - Generated predictions and evaluated performance
- Contributed to analysis of T5 model behavior across datasets

Student 2: Aryan Jain- 9923103046 (SciEntsBank Dataset & SBERT Model)

- Preprocessed the **SciEntsBank dataset**, including:
 - Cleaning and normalization of textual data
 - Mapping labels to appropriate score ranges

- Ensuring consistency with other datasets
- Implemented **Experiment 1 using SBERT**:
 - Generated sentence embeddings optimized for similarity
 - Computed similarity scores and evaluated results
- Implemented **Experiment 2**:
 - Built feature representations from SBERT embeddings
 - Applied regression model for score prediction
 - Performed evaluation and comparison
- Implemented **Experiment 3 (Fine-Tuning SBERT)**:
 - Fine-tuned the model on grading data
 - Evaluated improvements over baseline methods
- Established strong baseline performance using SBERT

Student 3: Astha Bansal- 9923103056 (Mohler Dataset, Pythia Model & System Design)

- Preprocessed the **Mohler dataset**, including:
 - Cleaning and structuring input data
 - Standardizing format across datasets
 - Preparing data for all experiments
- Implemented **Experiment 1 using Pythia**:
 - Generated embeddings from a decoder-only model
 - Computed similarity and evaluated results
- Implemented **Experiment 2**:
 - Created feature vectors using embeddings
 - Applied regression model
 - Evaluated model performance
- Implemented **Experiment 3 (Fine-Tuning Pythia)**:
 - Fine-tuned the model for grading
 - Compared results across experiments
- Designed and developed the **overall system pipeline**, including:
 - Dataset selection (Mohler, SciEntsBank, Beetle)
 - Project structure and modular code organization

Joint Contributions (Pragya Varshney, Aryan Jain, Astha Bansal)

All team members collaboratively contributed to:

- Writing the **project report**, including:
 - Literature review

- Methodology
 - Results and analysis
- Preparing the **final presentation (PPT)**
- Generating and interpreting:
 - Evaluation metrics
 - Graphs and visualizations (bar charts, heatmaps)
- Discussing:
 - Model performance
 - Comparative analysis
 - Key insights and conclusions

Overall Contribution

The division of tasks enabled parallel development across different datasets and models while maintaining a unified experimental framework. This collaborative approach ensured comprehensive evaluation, efficient workflow, and high-quality outcomes for the project.